Introduction to Cryptography

Lecture 37: RSA Encryption and Decryption Numericals

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Learning Outcomes

- Encrypt integer messages using RSA.
- Decrypt ciphertext using the private exponent.
- Apply repeated squaring for efficient modular powers.
- Verify RSA calculations with Python.

RSA Equations

$$\boxed{C = M^e \bmod n}$$

$$\boxed{M = C^d \bmod n}$$

Preconditions: $0 \leq M < n$; keys are correctly generated.

Given Textbook Example

$$p = 17,\ q = 11,\ n = 187,\ \phi(n) = 160$$

$$e = 7, \quad d = 23, \quad M = 88$$

$$PU = (7, 187), \quad PR = (23, 187)$$

Encryption: Break Down Exponent

$$C = 88^7 \bmod 187$$

$$7 = 4 + 2 + 1$$

$$88^2 \bmod 187 = 77$$

$$88^4 \bmod 187 = 77^2 \bmod 187 = 132$$

Encryption: Combine

$$88^7 \equiv 88^4 \cdot 88^2 \cdot 88 \pmod{187}$$

$$\equiv 132 \cdot 77 \cdot 88 \pmod{187}$$

$$\boxed{C = 11}$$

Decryption: Repeated Squares

$$M = 11^{23} \bmod 187, \quad 23 = 16 + 4 + 2 + 1$$

$$11^2 \bmod 187 = 121, \quad 11^4 \bmod 187 = 55$$

$$11^8 \bmod 187 = 33, \quad 11^{16} \bmod 187 = 154$$

Decryption: Combine

$$11^{23} \equiv 11^{16} \cdot 11^4 \cdot 11^2 \cdot 11 \pmod{187}$$
$$\equiv 154 \cdot 55 \cdot 121 \cdot 11 \pmod{187}$$
$$\boxed{M = 88}$$

Original plaintext recovered.

Repeated Squaring Principle

$$(ab) \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$$

Compute powers $a, a^2, a^4, a^8, \ldots$ modulo n, then multiply only powers required by the binary exponent.

Binary Exponentiation Example

$$13 = 8 + 4 + 1 = (1101)_2$$

$$a^{13} \bmod n = (a^8 \cdot a^4 \cdot a) \bmod n$$

Only a logarithmic number of squaring steps is required.

Python: RSA Numerical Check

Python 3

```
p, q, e, d = 17, 11, 7, 23
n = p * q
message = 88
ciphertext = pow(message, e, n)
recovered = pow(ciphertext, d, n)
print(ciphertext, recovered) # 11 88
```

Numerical 2: Generate and Use Keys

For $p = 5, q = 11, e = 3$:

$$n = 55, \quad \phi(n) = 40, \quad d = 27$$

Since $3 \times 27 = 81 \equiv 1 \pmod{40}$. Encrypt $M = 7$:

$$C = 7^3 \bmod 55 = \boxed{13}$$

Numerical 2: Decrypt

$$M = 13^{27} \bmod 55 = \boxed{7}$$

Verify in Python: `pow(13,27,55)` returns 7.

Caution

These tiny examples illustrate arithmetic only; they are not secure RSA implementations.

Exercise

Given $p = 13, q = 17, e = 5$, compute the ciphertext for $M = 10$ and recover the message.

$$n = 221, \quad d = 77$$

Exercise

Given $p = 13, q = 17, e = 5$, compute the ciphertext for $M = 10$ and recover the message.

$$n = 221, \quad d = 77$$

$$C = 10^5 \bmod 221 = \boxed{108}$$

$$108^{77} \bmod 221 = \boxed{10}$$

Common Errors

- Forgetting the modulus after intermediate multiplication.
- Choosing e without checking the GCD.
- Confusing public exponent e with private exponent d.
- Trying to compute giant integer powers directly.
- Mistaking textbook RSA for secure padded RSA.

Summary

$$\boxed{C = M^e \bmod n, \qquad M = C^d \bmod n}$$

Efficient implementation uses modular exponentiation.

Your Feedback Matters!

Please scan the QR code and share your feedback.



TCS-392 – Introduction to Cryptography